CTF Report

Full Name: Emmanuel Ateji
Program: HCS - Penetration Testing 1-Month Internship
Date: 9th March, 2025

Category: The World

Description: This category focuses on web application vulnerabilities, where the goal is to identify and exploit weaknesses in the application to uncover hidden flags. The challenge involves analyzing the structure of the webpage, testing for common vulnerabilities, and leveraging them to retrieve the flag.

Challenge Overview: The challenge presents a webpage with various functionalities, including input fields and potential hidden directories. The goal is to identify vulnerabilities such as improper input validation or directory traversal, exploit them, and retrieve the flag.

Steps for Finding the Flag:

1. **Initial Reconnaissance:** When first accessed, the challenge was presented with a seemingly normal webpage at https://the-world-web.hackatronics.com/ The visible content didn't provide any obvious clues, suggesting that the flag was hidden somewhere not immediately accessible through the user interface.

2. **Directory Enumeration:** Since there were no apparent input fields or obvious vulnerabilities in the visible application, there was a decision to perform directory and file enumeration:

- Launching DirBuster, a popular directory fuzzing tool
- Set the target URL to https://the-world-web.hackatronics.com/
- Selected a comprehensive wordlist to discover hidden files and directories

- Configured the tool to specifically look for files with a '.txt' extension, as text files are common places to store flags
- Started the enumeration process

3. **Analyzing the Results:** After running DirBuster for some time and carefully reviewed the results:

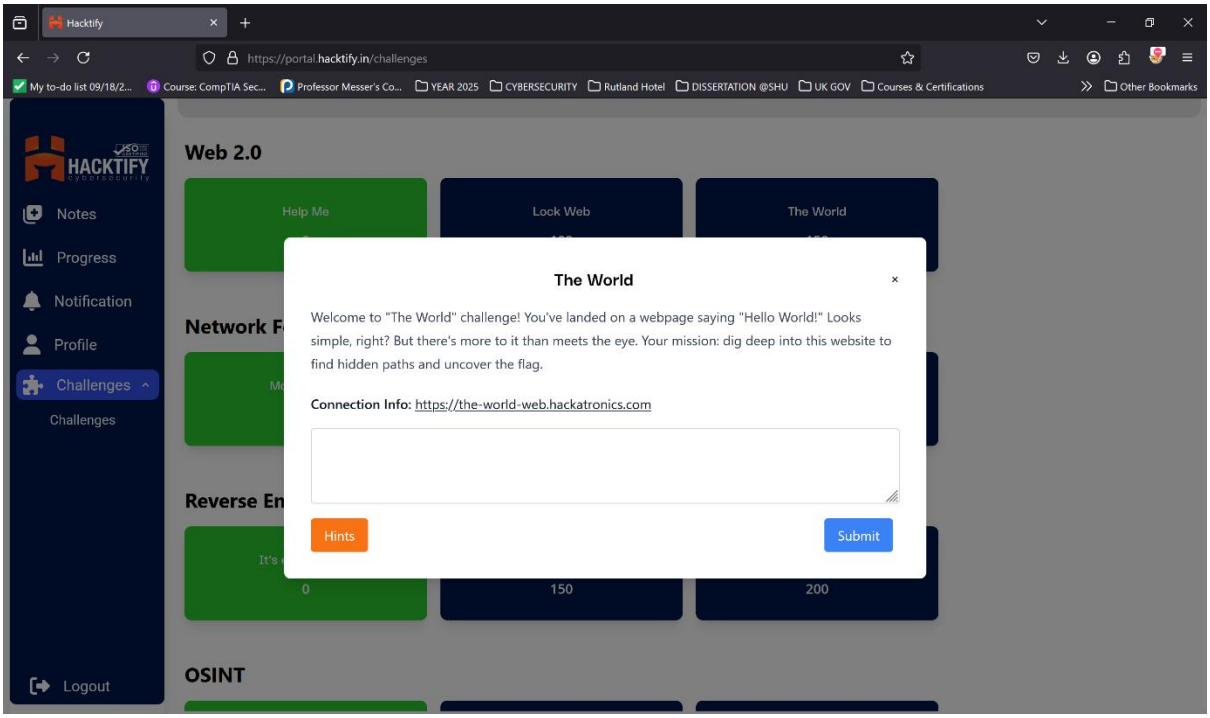
- Most attempted paths returned 404 Not Found responses
- Among the numerous results, I noticed one file that stood out: 'secret.txt'
- This file had a different response size compared to other 404 responses, indicating it actually existed on the server

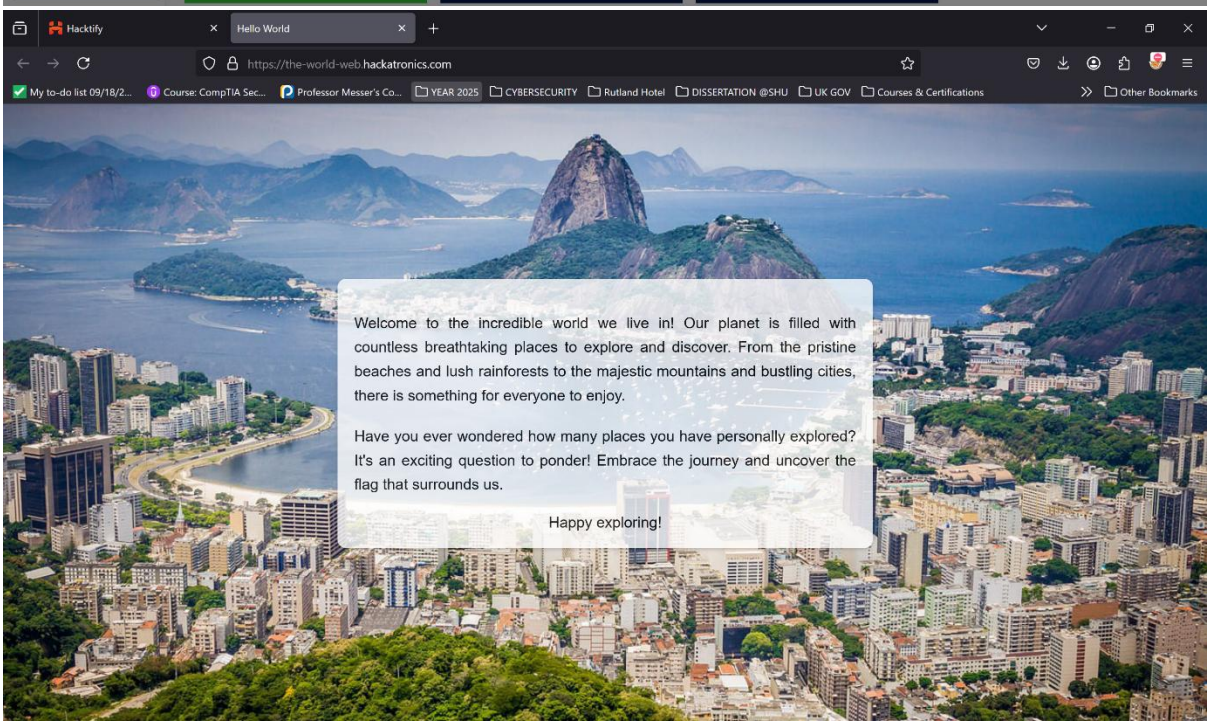
4. **Flag Retrieval:** With this discovery:

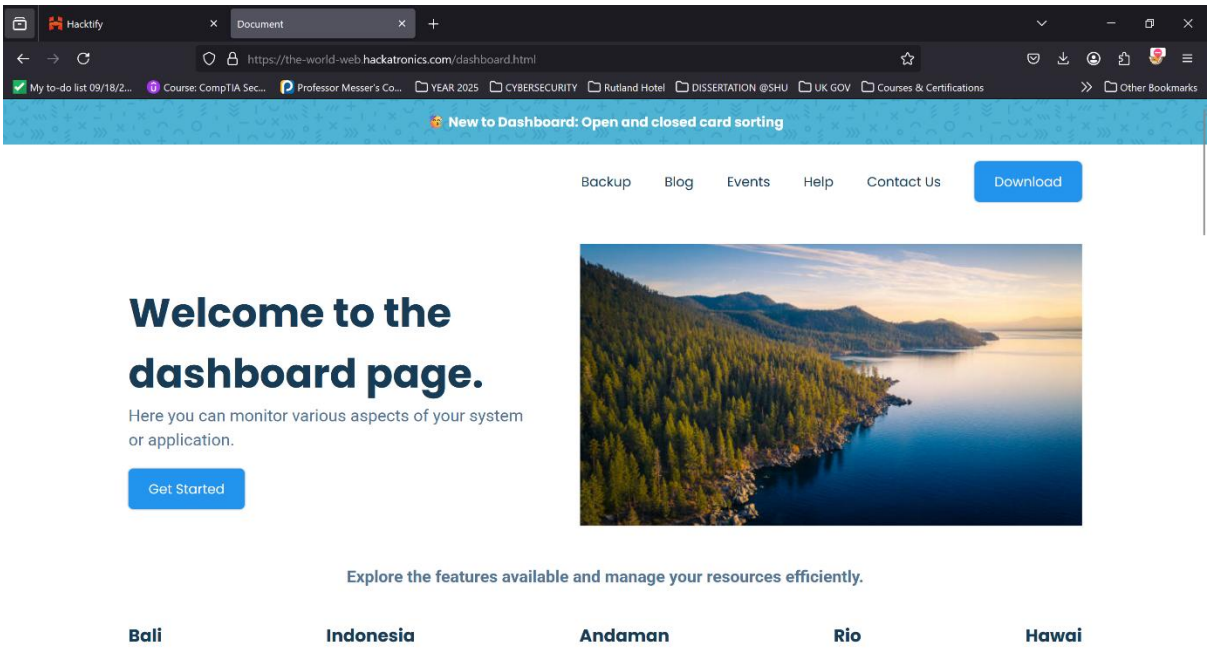
- Navigated directly to https://the-world-web.hackatronics.com/secret.txt in my browser
- Successfully accessed the file, which contained the encrypted flag, then used base64 to decode it.
- No further exploitation was necessary as the flag was displayed in plain text

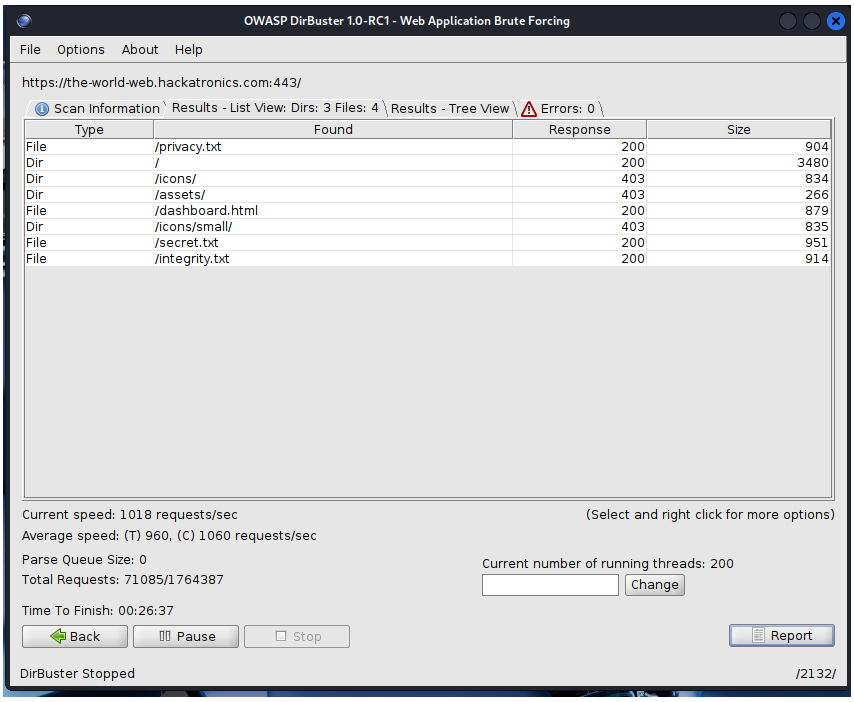
Flag: FLAG{Y0u_hav3_4xpl0reD_th3_W0rLd!}

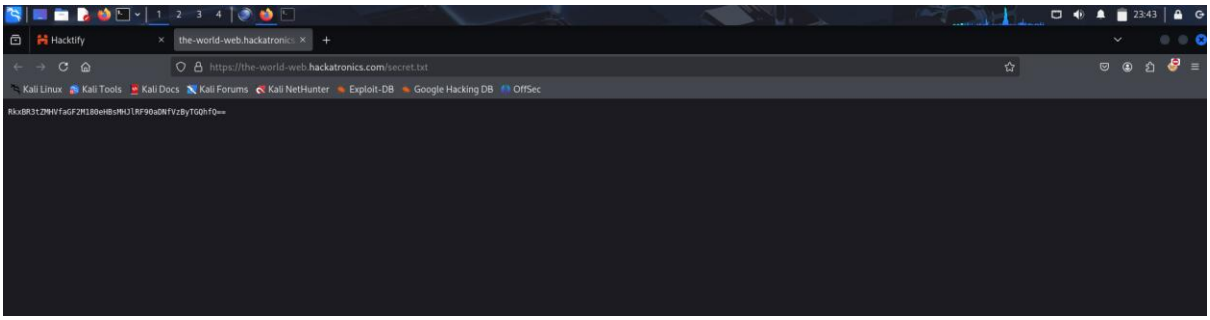
Proof of Concept

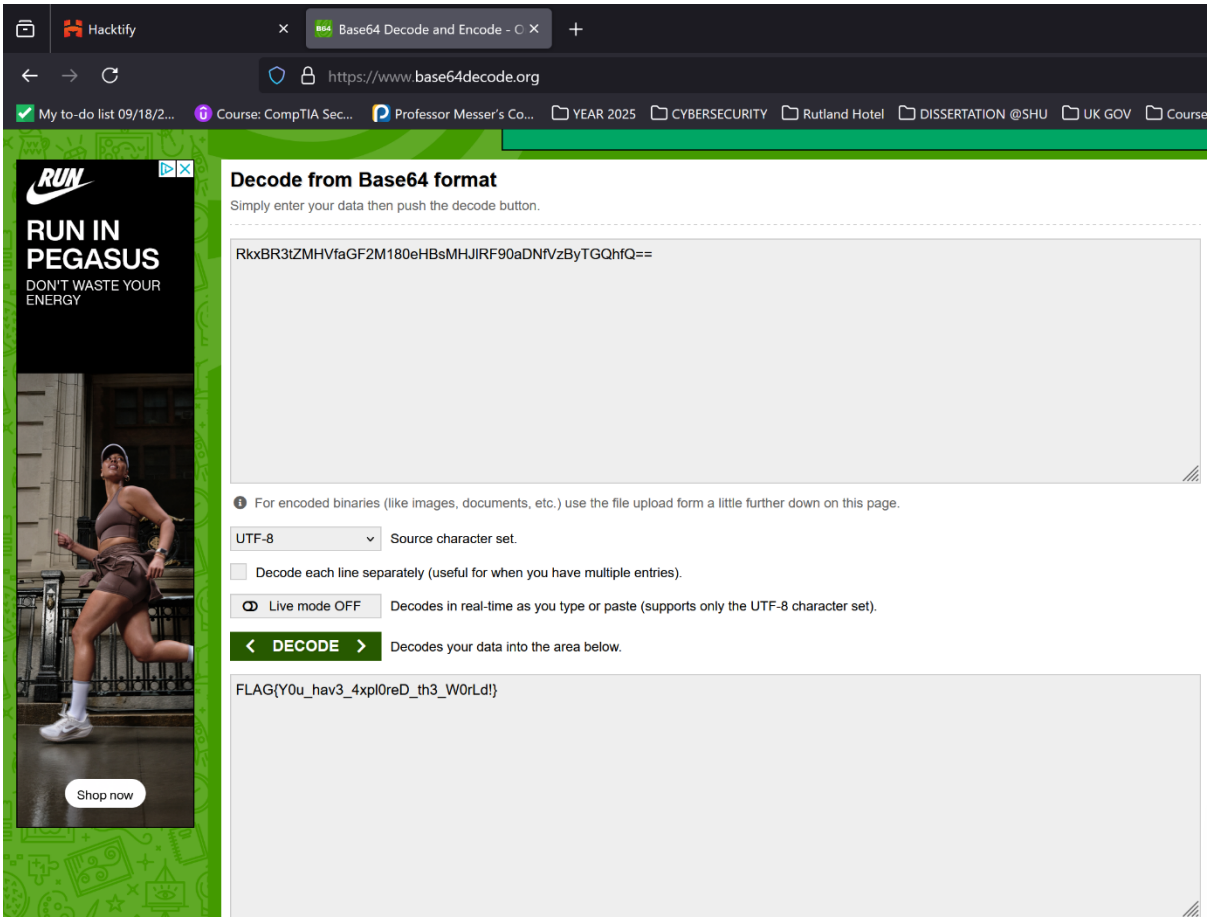

Hacktify
https://portal.hacktify.in/challenges
Web 2.0
Help Me
Lock Web
The World
Notes
Progress
Notification
Profile
Challenges
Challenges
Network F
Reverse En
0
150
200
OSINT
Logout
The World
Welcome to "The World" challenge! You've landed on a webpage saying "Hello World!" Looks simple, right? But there's more to it than meets the eye. Your mission: dig deep into this website to find hidden paths and uncover the flag.
Connection Info: https://the-world-web.hackatronics.com
Hints
Submit


Hacktify
Hello World
https://the-world-web.hackatronics.com
Welcome to the incredible world we live in! Our planet is filled with countless breathtaking places to explore and discover. From the pristine beaches and lush rainforests to the majestic mountains and bustling cities, there is something for everyone to enjoy.
Have you ever wondered how many places you have personally explored? It's an exciting question to ponder! Embrace the journey and uncover the flag that surrounds us.
Happy exploring!


Hacktify
Document
https://the-world-web.hackatronics.com/dashboard.html
New to Dashboard: Open and closed card sorting
Backup
Blog
Events
Help
Contact Us
Download
Welcome to the dashboard page.
Here you can monitor various aspects of your system or application.
Get Started
Explore the features available and manage your resources efficiently.
Bali
Indonesia
Andaman
Rio
Hawai


OWASP DirBuster 1.0-RC1 - Web Application Brute Forcing
File Options About Help
https://the-world-web.hackatronics.com:443/
Scan Information \ Results - List View: Dirs: 3 Files: 4 \ Results - Tree View \ Errors: 0 \
Type Found Response Size
File /privacy.txt 200 904
Dir / 200 3480
Dir /icons/ 403 834
Dir /assets/ 403 266
File /dashboard.html 200 879
Dir /icons/small/ 403 835
File /secret.txt 200 951
File /integrity.txt 200 914
Current speed: 1018 requests/sec
(Select and right click for more options)
Average speed: (T) 960, (C) 1060 requests/sec
Parse Queue Size: 0
Total Requests: 71085/1764387
Current number of running threads: 200
Change
Time To Finish: 00:26:37
Back
Pause
Stop
Report
DirBuster Stopped
/2132/


Hacktify
the-world-web.hackatronics
https://the-world-web.hackatronics.com/secret.txt
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OffSec


Hacktify
Base64 Decode and Encode - O
https://www.base64decode.org
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Course
RUN
RUN IN PEGASUS
DON'T WASTE YOUR ENERGY
Shop now
Decode from Base64 format
Simply enter your data then push the decode button.
RkxBR3tZMHVfaGF2M180eHBsMHJlRF90aDNfVzByTGQhfQ==
For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.
UTF-8
Source character set.
Decode each line separately (useful for when you have multiple entries).
Live mode OFF
Decodes in real-time as you type or paste (supports only the UTF-8 character set).
DECODE
Decodes your data into the area below.
FLAG{Y0u_hav3_4xpl0reD_th3_W0rLd!}

Category: Success Recipe

Description: CTF Challenges in "Success Recipe" category typically involve unconventional programming paradigms and esoteric languages that require creative problem-solving approaches.

Challenge Overview: "Success Recipe" presents what appears to be a nonsensical cooking recipe that contains encoded instructions. The challenge requires decoding the Brainfuck esoteric programming language to retrieve the flag.

Steps for Finding the Flag:

1. **Initial Reconnaissance:** When first opened, the challenge was presented with what looked like a bizarre recipe titled "Brain Meat" with unusual ingredient quantities and cooking instructions. It was noticed the file was named "recipe.txt-1740911941466-4663853" which didn't follow conventional naming patterns.

2. **Analysis of the Recipe Content:** Looking more closely at the recipe:

   - The title "Brain Meat" and the line "You really need to use your Brain to understand this" were hints about the Brainfuck programming language
   - The ingredients and cooking method were completely nonsensical (e.g., 91 cups of oregano, 93 cups of chillifakes)
   - The method contained repetitive instructions like "Put dijon mustard into mixing bowl" and "Crack the eggs until crack"

3. **Identifying the Programming Pattern**: It was noticed a pattern in the execution output with characters like +, -, >, <, [, and] - these are the fundamental commands in the Brainfuck programming language. The visualization showed a "Brainfuck Translator" that decoded the text to "yOu_40+_s3rv3d!".

4. **Understanding the Code Execution:** Looking at the Esolang Park Chef environment, it could be seen that:

- The code on the left side was executing the "recipe" instructions
- The right panel showed both the ingredients with their quantities and the execution output
- The execution output contained Brainfuck code (the pattern of +, -, >, <, [,])

5. **Decoding the Brainfuck:** The Brainfuck code when executed produced the string "yOu_40+_s3rv3d!" under the "Brainfuck Translator" interface and this needed to be wrapped in the flag format.

6. **Flag Retrieval:** As instructed where it says "Wrap the flag in flag{}", it was formatted the final result as: flag{yOu_40+_s3rv3d!}.

Flag: flag{yOu_40+_s3rv3d!}

Proof of Concept

HACKTIFY cybersecurity

esolangpark.vercel.app/ide/chef

Linkedin Opt | HACKTIFY INTERNSHIP | My to-do list 09/18/... | Course: CompTIA S... | Professor Messer's... | YEAR 2025 | CYBERSECURITY | Rutland Hotel | DISSERTATION @S... | All Bookmarks

Esolang Park | Chef | Run code | 20 ms | Read the esolang notes

Code Editor

```
Put the grapes into the mixing bowl.
Put the grapes into the mixing bowl.
Put the grapes into the mixing bowl.
Put the grapes into the mixing bowl.
Put chillifakes into mixing bowl.
Put salt into the mixing bowl.
Put the lard into the mixing bowl.
Put the lard into the mixing bowl.
Put the lard into the mixing bowl.
Put the lard into the mixing bowl.
Look the dragonfruits.
Put potatoes into the mixing bowl.
Look the dragonfruits until looked.
Put the grapes into the mixing bowl.
Smash the watermelons.
Put potatoes into the mixing bowl.
Smash the watermelons until smashed.
Put the grapes into the mixing bowl.
Put potatoes into the mixing bowl.
Put potatoes into the mixing bowl.
Put potatoes into the mixing bowl.
Put the grapes into the mixing bowl.
Put potatoes into the mixing bowl.
Put the grapes into the mixing bowl.
Put oregano into the mixing bowl.
Drink the water.
Put potatoes into the mixing bowl.
Drink the water until drinked.
Liquefy contents of the mixing bowl.
Pour contents of the mixing bowl into the baking dish.

Serves 1.
```

Visualization

End of program

Ingredients		Bowl 1	Dish 1
potatoes	43	46	
salt	45	45	
lard	60	45	
grapes	62	45	
dijon mustard	46	45	
oregano	91	45	
chillifakes	93	45	
eggs	0	45	
apples	0	45	
bananas	0	45	

User Input

Enter program input here...

Execution Output

```
++++++++++[>+>+++>+++++++>++++++++++
<<<<-]>>>>++++++++++++++++++++++.
<<+++++++++++++++++++.>>----.---------
-------------.<<++++.----.----
-.>>.++++++++++++++++++++.
<<++++++++.>>-.++++.<<.>>------------
------.<<-----------------.
```

md5decrypt.net/en/Brainfuck-translator/

Linkedin Opt | HACKTIFY INTERNSHIP | My to-do list 09/18/... | Course: CompTIA S... | Professor Messer's... | YEAR 2025 | CYBERSECURITY | Rutland Hotel | DISSERTATION @S... | All Bookmarks

- Home
- Encrypt / Decrypt
- Obfuscation
- Ciphers
- Conversion
- Network
- API
- Contact
- FR / EN

Brainfuck Translator

```
++++++++++[>+>+++>+++++++>++++++++++
<<<<-]>>>>++++++++++++++++++++++.
<<+++++++++++++++++++.>>----.----------------------.
<<++++.----.-----.>>.++++++++++++++++++++.
<<++++++++.>>-.++++.<<.>>-----------------.<<----------
--------.
```

```
y0u_4O+_s3rv3d!
```

Argument(s)

Encode | Decode

Category: Shadow Web

Description:

Challenge Overview: The "Shadow Web" challenge requires analyzing network packet captures to identify hidden data within HTTP requests. The PCAP file must be examined to find concealed messages transmitted during web communication.

Steps for Finding the Flag:

1. **Initial Reconnaissance:**
    - The provided Wireshark capture shows multiple HTTP POST requests from 192.168.64.135 to 192.168.1.1
    - All requests use HTTP/1.1 with Content-Type: multipart/form-data and a WebKitFormBoundary
    - There are 36+ sequential HTTP POST requests visible in the capture

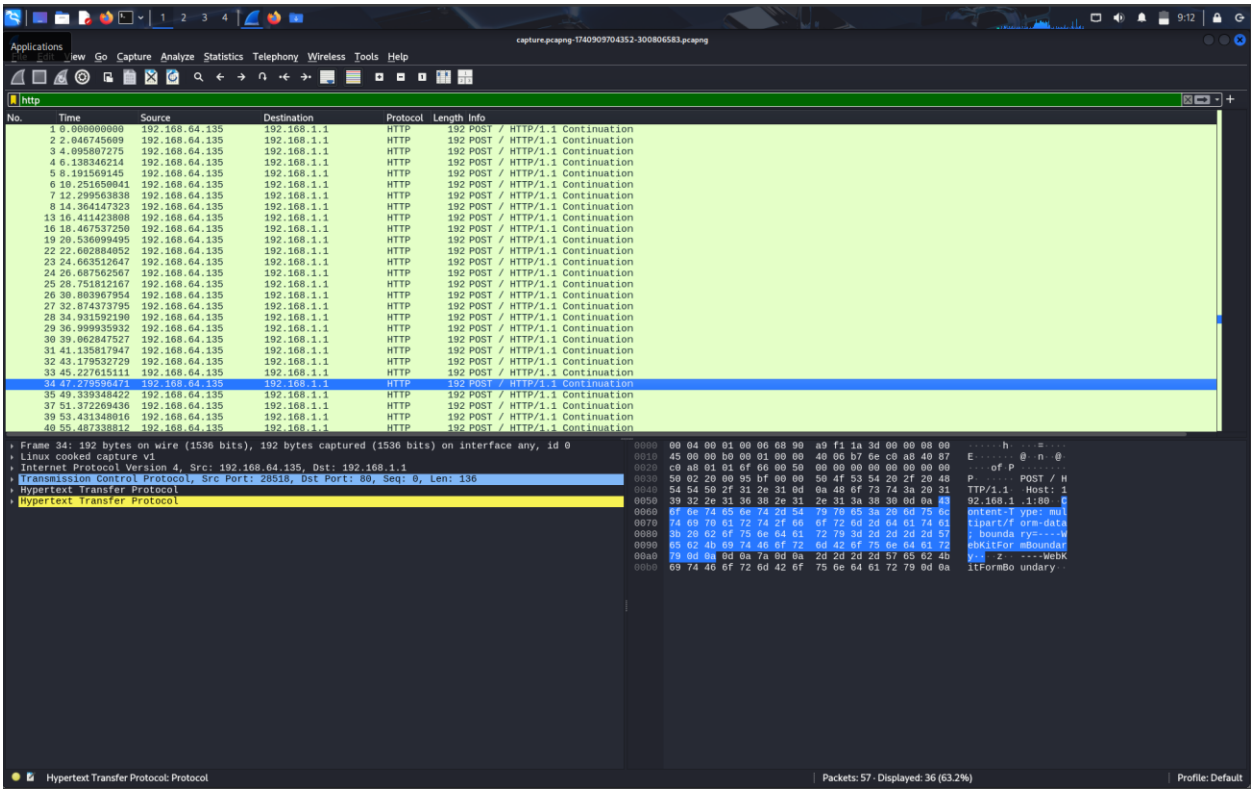
2. **Traffic Analysis:**
    - When examining the HTTP stream, we can see the requests contain form data with a WebKitFormBoundary delimiter
    - The data appears to be minimal, possibly containing single characters
    - All requests are coming from the same source IP (192.168.64.135) to the same destination (192.168.1.1:80)

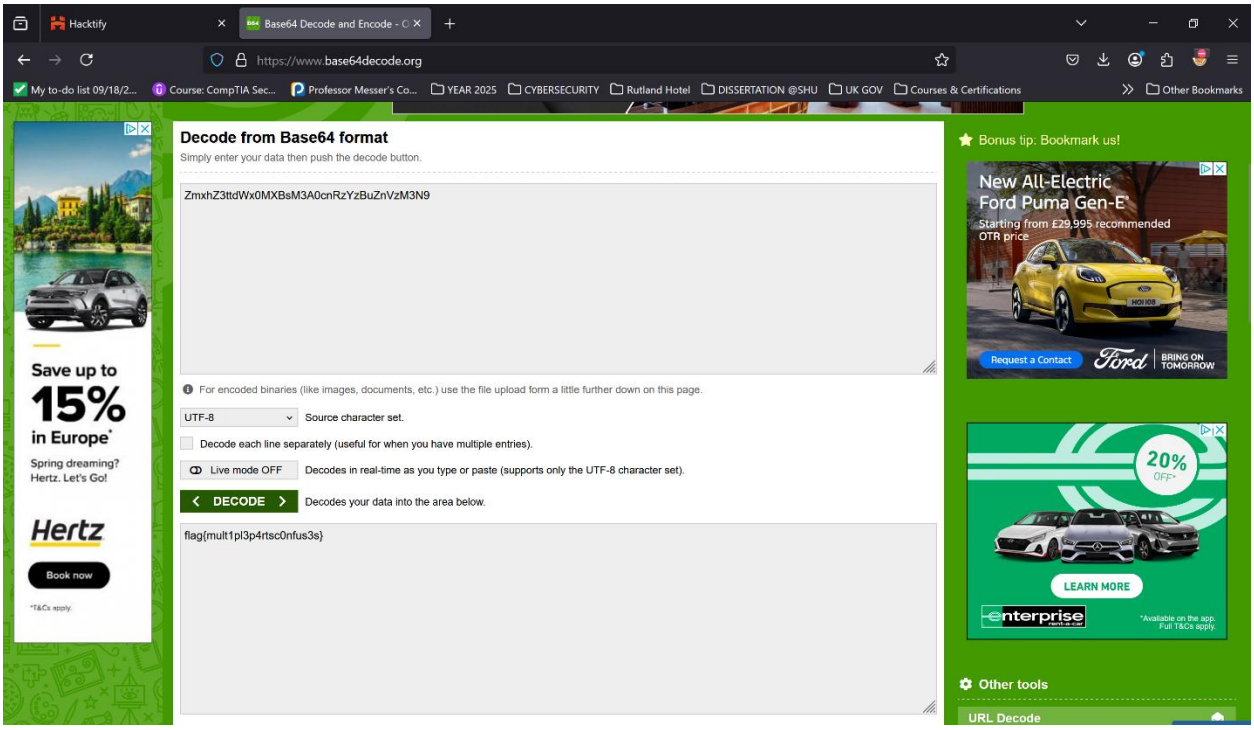
3. **Data Extraction:**
    - By filtering for HTTP traffic and examining each request sequentially, we can extract individual characters from each request
    - There appears to be a character "Z" visible in one of the requests
    - The pattern suggests each HTTP POST request contains a single character

4. **Flag Retrieval:**

- The extracted characters form a Base64 encoded string: "ZmxhZ3itdWx0MXBsM3A0cnRzc0nfus3s"
- Using a Base64 decoder, the encoded string decodes to: "flag{mult1pl3p4rtsc0nfus3s}"

Flag: flag{mult1pl3p4rtsc0nfus3s}





Shadow web

Unravel hidden data within the intricate landscape of protocols. This MULTIverse of packets contains some Form Data which can reveal the secrets of Web. Try to find this secrets that are scattered to get a flag.

flag{mult1pl3p4rtsc0nfus3s}

Files:

- capture.pcapng-1740909704352-300806583.pcapng

Hints

Submit

Wrong answer, try again.

Category: Wh@t7he####

Description: Esoteric Programming Languages

Challenge Overview: This challenge presents the player with a piece of code written in the ReverseFuck esoteric programming language. The objective is to decode this language to retrieve the hidden flag.

Steps for Finding the Flag:

1. **Initial Reconnaissance:** Upon examining the file for the challenge, a string of symbols consisting of plus signs, minus signs, angle brackets, and commas is presented. This pattern is characteristic of an esoteric programming language.
2. **Language Identification**: After analyzing the syntax pattern with sequences of "+", "-", "<", ">", and "," characters, it can be identified as ReverseFuck, an esoteric programming language based on Brainfuck.
3. **Tool Selection:** To interpret the ReverseFuck code, a specialized tool is needed. The dcode.fr website offers a ReverseFuck interpreter that can execute this type of code.
4. **Code Execution:** The ReverseFuck code is pasted into the interpreter section of the dcode.fr website. The interpreter processes the code according to the ReverseFuck specification.

5. **Flag Retrieval:** After executing the code through the interpreter, the output reveals the flag: flag{R3v3rs3ddd_70_g3t_m3}.

Flag: flag{R3v3rs3ddd_70_g3t_m3}

Proof of Concept

File Edit View

```
----------]<-<---<-------<---------->>>>+[<<<<--,------,
++++++++++,------,--------------------,>------------,>---------------------,<<+++++,
++++++++++++++++,-------------,-,>>,<<++++++++++++++,,,+++++,>>----,
+++++++,<<,--------,>>---,<<-------------,>-------------,<+++++++,>>,<<----------------,
```

Wh@t7he####

×

Change my mind!

Flag format: flag{!@#$%^&*()_+}

flag{R3vers3ddd_70_g3t_m3}

Files:

- chall.file-1740911737469-67832877.file

Hints

Submit

Category: Lock Web

Description: This category focuses on exploiting vulnerabilities in web applications, requiring knowledge of web technologies, common security flaws, and enumeration techniques.

Challenge Overview: Lock Web presents a digital lock interface that requires bypassing authentication to gain access to protected content. The challenge tests skills in web enumeration, information disclosure analysis, and PIN cracking techniques.

Steps for Finding the Flag:

1. **Initial Reconnaissance:** The target website displayed a numeric keypad interface at https://lock-web-web.hackatronics.com, resembling a digital lock system. Using OWASP DirBuster tool revealed the site structure, showing a root directory and a robots.txt file. The DirBuster scan indicated 28 errors during the enumeration process while successfully identifying these two resources.

2. **Directory Enumeration:** Examination of the robots.txt file revealed critical information about the web application. The file contained several key-value pairs including:
    - buildNumber: "v20190816"
    - debug: false
    - modelName: "Valencia"
    - correctPin: "1928"

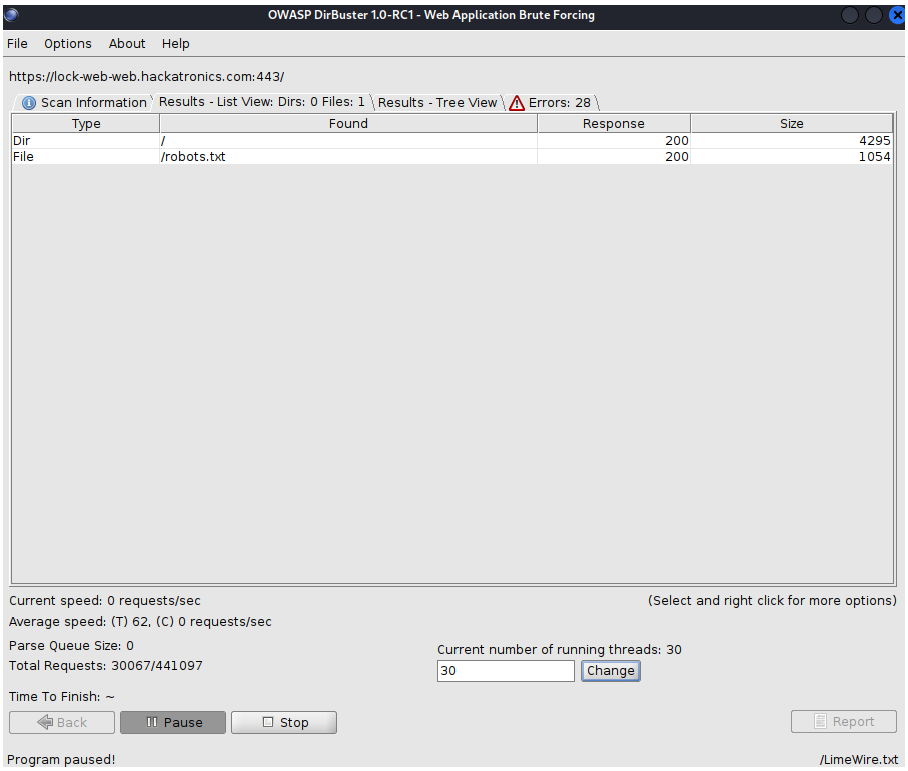
   This was a significant finding as it exposed the PIN code required to unlock the system, demonstrating poor security practices where sensitive authentication data was stored in a publicly accessible file.

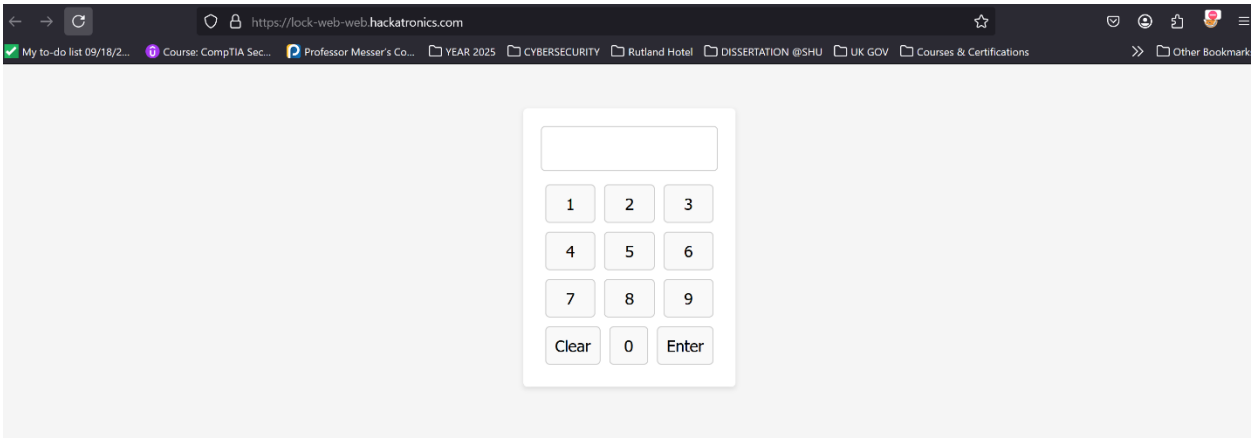
3. **Input Validation Testing:** With the PIN code "1928" discovered in the robots.txt file, the lock interface was tested using this value. The numeric keypad on the website allowed for input of the discovered PIN. When entered correctly, the system accepted the PIN and processed the authentication request.

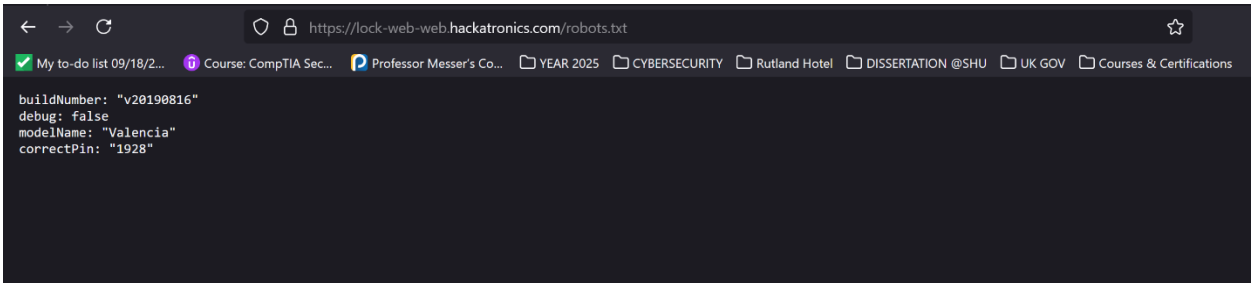
4. **Exploitation:** After entering the PIN "1928" into the keypad interface, the system authenticated the user and displayed a success message. This confirmed that the PIN discovered in the robots.txt file was valid and could be used to bypass the authentication mechanism.

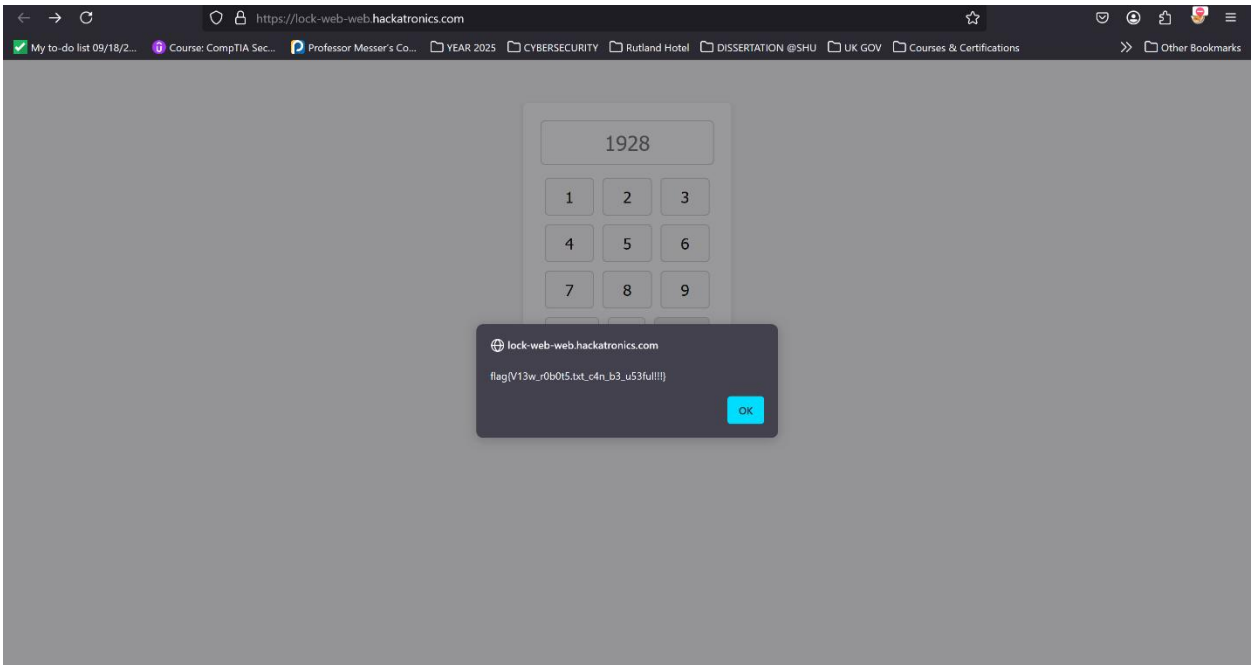
5. **Flag Retrieval:** Upon successful authentication with the PIN "1928", the system revealed the flag in a popup message. The flag was displayed as: flag{v13w_r0b0t5.txt_c4n_b3_u53full!}

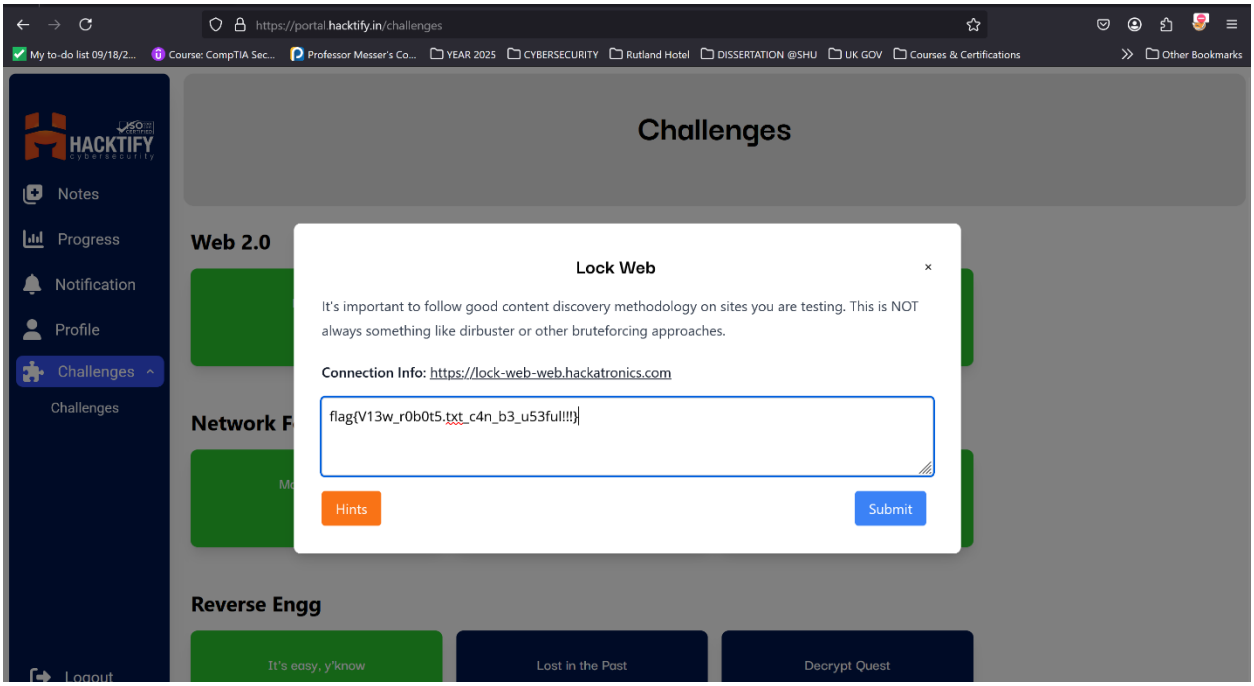
Flag: flag{v13w_r0b0t5.txt_c4n_b3_u53full!}


OWASP DirBuster 1.0-RC1 - Web Application Brute Forcing
File Options About Help
https://lock-web-web.hackatronics.com:443/
Scan Information Results - List View: Dirs: 0 Files: 1 Results - Tree View Errors: 28
Type Found Response Size
Dir / 200 4295
File /robots.txt 200 1054
Current speed: 0 requests/sec
(Select and right click for more options)
Average speed: (T) 62, (C) 0 requests/sec
Parse Queue Size: 0
Total Requests: 30067/441097
Current number of running threads: 30
30
Change
Time To Finish: ~
Back
Pause
Stop
Report
Program paused!
/LimeWire.txt


https://lock-web-web.hackatronics.com
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
1 2 3
4 5 6
7 8 9
Clear 0 Enter


https://lock-web-web.hackatronics.com/robots.txt
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
buildNumber: "v20190816"
debug: false
modelName: "Valencia"
correctPin: "1928"


https://lock-web-web.hackatronics.com
Other Bookmarks
1928
1 2 3
4 5 6
7 8 9
lock-web-web.hackatronics.com
flag{V13w_r0b0t5.txt_c4n_b3_u53ful!!!}
OK


https://portal.hacktify.in/challenges
Other Bookmarks
Notes
Progress
Notification
Profile
Challenges
Challenges
Logout
Challenges
Web 2.0
Network F
Reverse Engg
It's easy, y'know
Lost in the Past
Decrypt Quest
Lock Web
It's important to follow good content discovery methodology on sites you are testing. This is NOT always something like dirbuster or other bruteforcing approaches.
Connection Info: https://lock-web-web.hackatronics.com
flag{V13w_r0b0t5.txt_c4n_b3_u53ful!!!}
Hints
Submit

Category: Corrupted

Description: The challenge was centered around file forensics, specifically dealing with a corrupted picture file. The purpose was to fix the image and retrieve the concealed flag.

Challenge Overview: The challenge included a corrupted PNG image (chall.png-1740909420074-618633075.png). The goal was to analyze and restore the image in order to obtain the concealed flag.

Steps for Finding the Flag:

1. **Initial Reconnaissance:**
    - Downloaded the provided corrupted PNG file.
    - Attempted to open it using standard image viewers (e.g., Windows Photo Viewer, GIMP).
    - Observed that the image was not displaying correctly, indicating file corruption.
2. **Analyzing the data:**
    - Identified that the issue was likely related to a corrupt file header.
    - Checked the file structure using Hex Editors but opted for an automated repair method.
3. **Repairing the Corrupted Image:**
    - Used the online tool EaseUS Photo Repair to repair the image:
    - Website: https://repair.easeus.com/photo_repair/#upload
    - Successfully restored the image.
4. **Extracting the Flag:**

- After repairing, opened the image and observed the hidden flag text.

5. **Flag Retrieval:** The flag was clearly visible in the repaired image.

Flag: flag{m3ss3d_h3ad3r$}

Proof of concept

Corrupted

This is too EasyPeasyy!

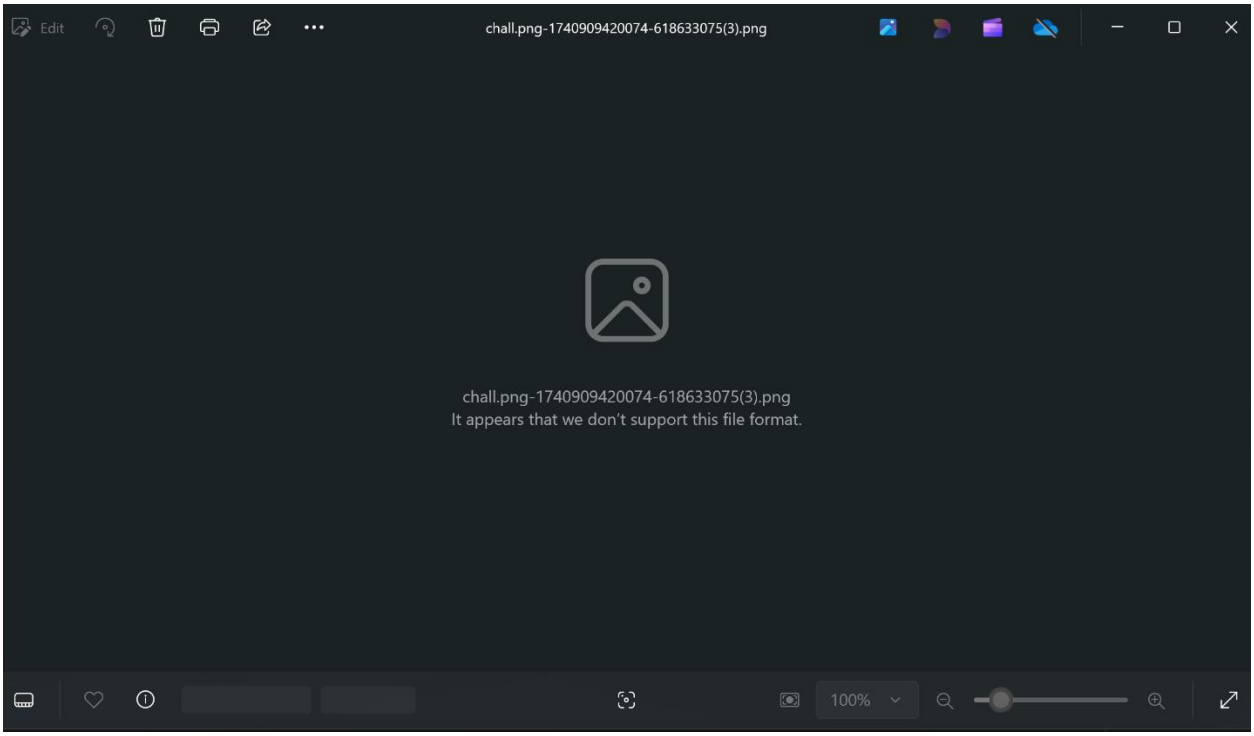
Flag format: flag{!@#$%^&*()_+}

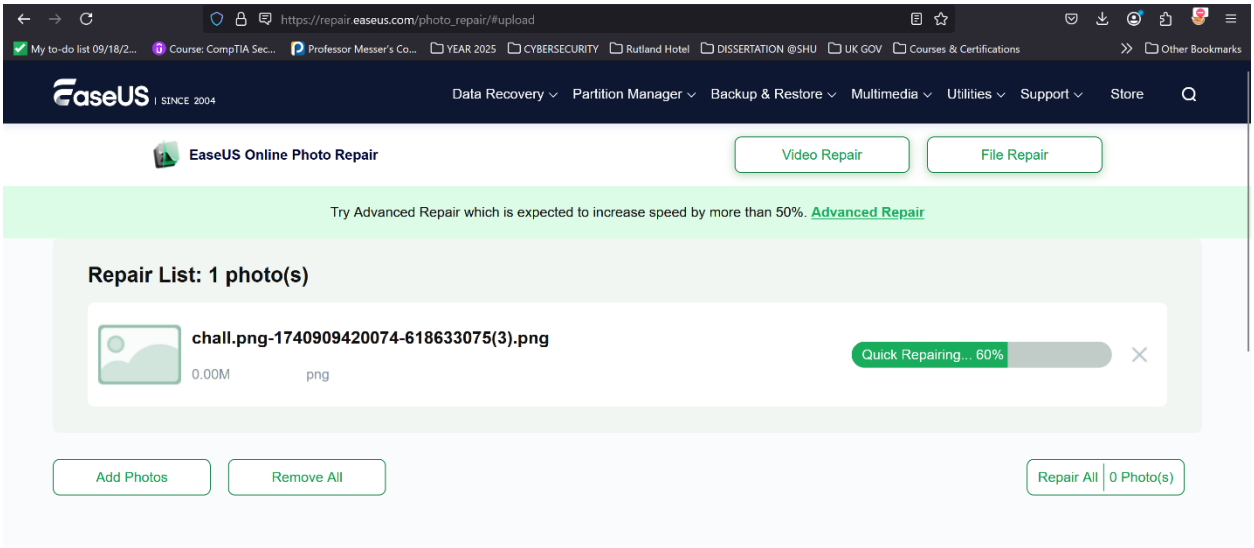
Files:

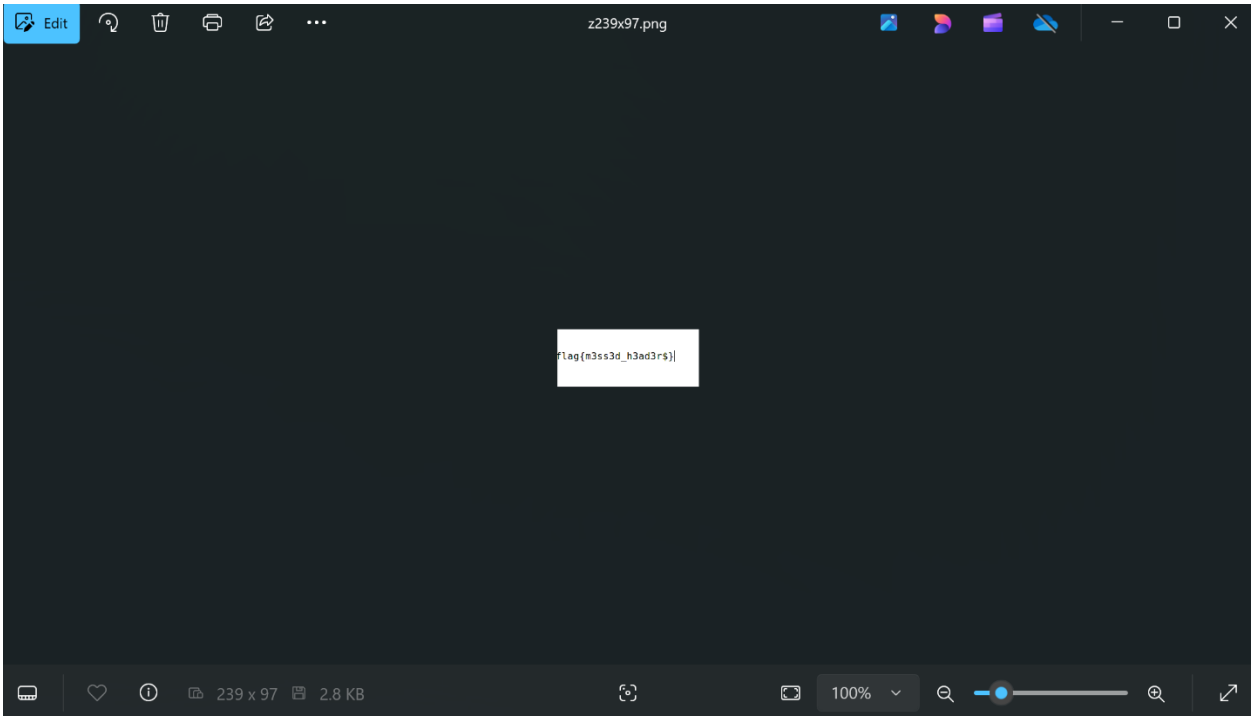
- chall.png-1740909420074-618633075.png

Hints

Submit




https://repair.easeus.com/photo_repair/#upload
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
EaseUS | SINCE 2004
Data Recovery
Partition Manager
Backup & Restore
Multimedia
Utilities
Support
Store
EaseUS Online Photo Repair
Video Repair
File Repair
Try Advanced Repair which is expected to increase speed by more than 50%. Advanced Repair
Repair List: 1 photo(s)
chall.png-1740909420074-618633075(3).png
0.00M
png
Quick Repairing... 60%
Add Photos
Remove All
Repair All
0 Photo(s)


Edit
z239x97.png
flag{m3ss3d_h3ad3r$}
239 x 97
2.8 KB
100%

HACKTIFY cybersecurity

Corrupted

×

This is too EasyPeasyy!

Flag format: flag{!@#$%^&*()_+}

flag{m3ss3d_h3ad3r$}

Files:

- chall.png-1740909420074-618633075.png

Hints

Submit